Pontifícia Universidade Católica
do Rio de Janeiro



Gabriel Lobo de Sant'Anna Rosas

Dissertação de Mestrado

Dissertação apresentada como requisito parcial para obtenção do grau de Mestre pelo Programa de Pós–graduação em Informática, do Departamento de Informática da PUC-Rio.

Orientador : Adriano Francisco Branco
Co-orientador: Jonatas dos Santos Grosman

Rio de Janeiro
de

Pontifícia Universidade Católica
do Rio de Janeiro



Gabriel Lobo de Sant'Anna Rosas

Dissertação apresentada como requisito parcial para obtenção do grau de Mestre pelo Programa de Pós–graduação em Informática da PUC-Rio. Aprovada pela Comissão Examinadora abaixo:

Adriano Francisco Branco
Orientador
Departamento de Informática – PUC-Rio

Jonatas dos Santos Grosman
Co-orientador
Departamento de Informática – PUC-Rio

Rio de Janeiro, de de

Todos os direitos reservados. A reprodução, total ou parcial do trabalho, é proibida sem a autorização da universidade, do autor e do orientador.

Gabriel Lobo de Sant'Anna Rosas

Graduado em ciência da computação pela Universidade de Harvard.

Ficha Catalográfica

Lobo de Sant'Anna Rosas, Gabriel

/ Gabriel Lobo de Sant'Anna Rosas; orientador: Adriano Francisco Branco; co-orientador: Jonatas dos Santos Grosman. – .

27 f: il. color. ; 30 cm

Dissertação (mestrado) - Pontifícia Universidade Católica do Rio de Janeiro, Departamento de Informática, .

Inclui bibliografia

I. Francisco Branco, Adriano. II. dos Santos Grosman, Jonatas. III. Pontifícia Universidade Católica do Rio de Janeiro. Departamento de Informática. IV. Título.

CDD:

Agradecimentos

Resumo

Lobo de Sant'Anna Rosas, Gabriel; Francisco Branco, Adriano; dos Santos Grosman, Jonatas. . Rio de Janeiro, . 27p. Dissertação de Mestrado – Departamento de Informática, Pontifícia Universidade Católica do Rio de Janeiro.

Palavras-chave

.

Abstract

Lobo de Sant'Anna Rosas, Gabriel; Francisco Branco, Adriano (Advisor); dos Santos Grosman, Jonatas (Co-Advisor). . Rio de Janeiro, . 27p. Dissertação de Mestrado – Departamento de Informática, Pontifícia Universidade Católica do Rio de Janeiro.

The acquisition of triangular meshes typically introduces undesired noise...

Keywords

.

Sumário

1 Introdução **12**
1.1 Motivação 12
1.2 Definição do problema 13
1.3 Justificativa 13
1.4 Escopo e delimitação 14
1.5 Organização do documento 14

2 Situação Atual **15**
2.1 Conceitos e terminologia 15
2.2 Abordagens para previsão de carga 19
2.3 Abordagens para alocação de recursos 21
2.4 Comparativo dos trabalhos relacionados 23
2.5 Tecnologias Utilizadas 24

3 Referências bibliográficas **26**

Lista de figuras

Lista de tabelas

Tabela 2.1 Comparativo entre os trabalhos relacionados e a proposta deste trabalho. 24

Lista de algoritmos

Lista de Códigos

1
Introdução

1.1
Motivação

A computação em nuvem consolidou-se como a principal forma de disponibilizar infraestrutura computacional para empresas e instituições. Nesse modelo, o provedor mantém grandes centros de dados e precisa decidir, de forma contínua, quanto de processamento, memória e armazenamento deve estar disponível para atender à demanda de seus clientes. Como a demanda tende a crescer, impulsionada sobretudo por aplicações de Inteligência Artificial e de análise de grandes volumes de dados, a gestão eficiente desses recursos tornou-se um fator central para a sustentabilidade econômica dos provedores (ROSSI et al., 2023).

A dificuldade está no fato de que a carga de trabalho não é constante. Ela apresenta picos, padrões periódicos e variações aparentemente aleatórias. Diante disso, a abordagem mais comum é a alocação reativa, na qual o sistema ajusta os recursos apenas depois de observar uma mudança na demanda, normalmente por meio de regras de limiar sobre a utilização. Como a reação ocorre com atraso, esse tipo de estratégia tende a degradar o desempenho durante os picos e a desperdiçar recursos nos períodos de baixa utilização (BODRA; KHAIRNAR, 2025).

Esse cenário cria um dilema permanente. Manter recursos em excesso (superprovisionamento) garante a qualidade do serviço, mas eleva o custo operacional. Manter recursos de menos (subprovisionamento) reduz o custo, mas provoca violações dos Acordos de Nível de Serviço (*Service Level Agreements*, SLA) firmados com os clientes. Uma alternativa é a alocação preditiva, que utiliza o histórico de uso para antecipar a demanda futura e provisionar recursos antes que eles sejam necessários (BODRA; KHAIRNAR, 2025). Nesse contexto, prever a carga com precisão passa a ser uma pré-condição para otimizar a alocação (ROSSI et al., 2023).

A previsão de carga em nuvem foi tradicionalmente tratada com modelos estatísticos lineares, como o ARIMA (CALHEIROS et al., 2015). Mais recentemente, redes neurais profundas, em especial as arquiteturas recorrentes, passaram a ser aplicadas ao problema com bons resultados, inclusive em abordagens multivariadas que consideram simultaneamente o uso de processador e de memória (XU et al., 2022).

1.2 Definição do problema

O problema tratado neste trabalho tem duas partes encadeadas. A primeira é um problema de previsão: dado o histórico de utilização de recursos de um *cluster*, amostrado em intervalos regulares, estimar a demanda de processador e de memória em um horizonte curto. A segunda é um problema de decisão: a partir dessa estimativa, determinar quantas instâncias de recursos (máquinas virtuais ou contêineres) devem permanecer ativas em cada intervalo.

De forma preliminar, a decisão pode ser formulada como a minimização do custo operacional sujeita a uma restrição de qualidade de serviço:

$$\min_{n_1,\ldots,n_T} \sum_{t=1}^{T} c \cdot n_t \quad \text{sujeito a} \quad \frac{1}{T}\sum_{t=1}^{T} \mathbf{1}\left[\, d_t > k \cdot n_t \,\right] \leq \varepsilon, \tag{1-1}$$

em que n_t é o número de instâncias ativas no intervalo t, c é o custo de uma instância por intervalo, k é a capacidade de uma instância, d_t é a demanda real observada, $\mathbf{1}[\cdot]$ é a função indicadora e ε é a fração máxima de intervalos com violação de SLA tolerada. Como d_t não é conhecida no momento da decisão, a escolha de n_t depende da previsão $\hat{d}_t$, e a qualidade dessa previsão afeta diretamente o custo e o número de violações. Essa formulação será refinada no Capítulo **??**.

1.3 Justificativa

A relevância do problema é econômica e operacional. Cada ponto percentual de recursos ociosos representa custo para o provedor, e cada violação de SLA representa perda de qualidade percebida pelo cliente e, eventualmente, penalidades contratuais.

Apesar do volume de trabalhos sobre previsão de carga, a maior parte deles avalia os modelos apenas por métricas de erro pontual, como RMSE e MAPE. Rossi et al. (2023) mostram que essas métricas, isoladamente, não oferecem uma visão clara da utilidade dos modelos para a gestão de recursos, e apontam que integrar os modelos de previsão aos mecanismos de alocação em um mesmo fluxo ainda é uma questão em aberto. O esDNN (XU et al., 2022) chega a aplicar suas previsões em uma simulação de escalonamento automático de máquinas, mas compara o resultado apenas com uma estratégia baseada na média das observações anteriores, sem confrontá-lo com outros métodos de previsão sob a mesma política de alocação.

Do lado da alocação, a revisão de Bodra e Khairnar (2025) observa que a literatura carece de comparações quantitativas sistemáticas entre abordagens e que técnicas de aprendizado por reforço profundo, embora promissoras, são

sensíveis ao projeto da função de recompensa e aos hiperparâmetros, o que dificulta sua adoção confiável. Isso favorece abordagens em que a previsão e a decisão sejam componentes separados, mais fáceis de interpretar e de avaliar.

Dessa forma, há espaço para um trabalho que avalie de ponta a ponta o efeito da previsão sobre a alocação, medindo custo e violações de SLA, e que compare redes neurais e métodos tradicionais sob as mesmas condições, utilizando dados públicos de *clusters* reais.

1.4 Escopo e delimitação

Para que o trabalho seja viável no tempo disponível, o escopo é delimitado da seguinte forma.

Incluído no escopo. O trabalho utilizará rastros públicos de utilização de *clusters*, especificamente o Alibaba Cluster Trace (Alibaba Group, 2018) e o Google Cluster Trace (WILKES, 2020), agregados em séries temporais de uso de processador e de memória. Serão considerados horizontes de previsão curtos, compatíveis com decisões de alocação na escala de minutos. A alocação será avaliada em um centro de dados simulado, com um modelo de custo simplificado baseado no custo por instância ativa e uma métrica de violação de SLA baseada na demanda não atendida. A abordagem proposta, baseada em rede neural, será comparada com uma política reativa por limiar e com métodos tradicionais de previsão.

Fora do escopo. Não fazem parte deste trabalho a implantação em uma infraestrutura de nuvem real, o posicionamento detalhado de máquinas virtuais em servidores físicos e a migração entre eles, a otimização de consumo de energia, cenários com múltiplos provedores e o uso de aprendizado por reforço para a tomada de decisão.

1.5 Organização do documento

O restante deste documento está organizado da seguinte forma. O Capítulo 2 apresenta a terminologia básica e as principais abordagens existentes para previsão de carga e alocação de recursos em nuvem, com suas limitações, além das tecnologias que serão utilizadas. O Capítulo **??** descreve a solução proposta, seus objetivos específicos e o que será efetivamente implementado. O Capítulo **??** revisa o plano de ação, comparando as atividades propostas com as realizadas. Por fim, o Capítulo **??** apresenta os cronogramas do Projeto Final I e do Projeto Final II.

2
Situação Atual

Este capítulo apresenta o contexto necessário para compreender o problema tratado neste trabalho. A Seção 2.1 define os conceitos e a terminologia básica. As Seções 2.2 e 2.3 descrevem, respectivamente, as abordagens existentes para previsão de carga e para alocação de recursos em nuvem, destacando suas limitações. A Seção 2.4 compara os trabalhos mais próximos desta proposta, e a Seção 2.5 apresenta as tecnologias que serão utilizadas.

2.1
Conceitos e terminologia

2.1.1
Elasticidade, carga de trabalho e séries temporais

Uma das características essenciais da computação em nuvem é a elasticidade rápida, isto é, a capacidade de provisionar e liberar recursos rapidamente, em alguns casos de forma automática, acompanhando a demanda (MELL; GRANCE, 2011). Herbst, Kounev e Reussner (2013) definem elasticidade como o grau em que um sistema se adapta a mudanças na carga de trabalho, provisionando e liberando recursos de forma autônoma, de modo que, a cada instante, os recursos disponíveis correspondam o mais fielmente possível à demanda corrente. Essa definição deixa claro que a elasticidade tem duas falhas possíveis: oferecer recursos de mais ou recursos de menos.

A carga de trabalho (*workload*) de um *cluster* é o conjunto de demandas submetidas a ele, como tarefas em lote, serviços de longa duração e requisições de usuários. Do ponto de vista da infraestrutura, essa carga se manifesta como utilização de recursos, principalmente processador e memória. Quando essa utilização é agregada em intervalos regulares de duração Δ, obtém-se uma série temporal $y_1, y_2, \ldots, y_t$, em que y_t é a utilização média (ou máxima) no intervalo t. Séries de carga em nuvem costumam combinar tendência, padrões periódicos (por exemplo, diários e semanais), ruído e picos abruptos, o que torna sua previsão difícil (XU et al., 2022).

Prever a carga significa estimar $\hat{y}_{t+h}$, o valor da série h intervalos à frente, a partir de uma janela com as w observações mais recentes. O parâmetro h é o horizonte de previsão e w é o tamanho da janela de entrada. A previsão é univariada quando usa apenas a própria série e multivariada quando incorpora outras séries correlacionadas, como prever o uso de processador a partir do histórico de processador e de memória (XU et al., 2022).

Pesquisas sobre o tema dependem de rastros (*traces*) públicos de *clusters* reais, que registram eventos e uso de recursos de máquinas, tarefas e contêineres. O rastro mais recente do Google cobre oito células do gerenciador Borg durante o mês de maio de 2019 (WILKES, 2020). O rastro de 2018 da Alibaba cobre cerca de 4.000 máquinas de um *cluster* de produção ao longo de oito dias, com serviços *online* e tarefas em lote compartilhando as mesmas máquinas, e inclui tabelas de uso de recursos por máquina e por contêiner (Alibaba Group, 2018). Como esses rastros registram eventos e medições em granularidade fina, transformá-los em séries temporais regulares é uma etapa de pré-processamento necessária, que faz parte deste trabalho.

2.1.2 Acordos de nível de serviço

A qualidade de um serviço em nuvem é expressa por três conceitos relacionados (BEYER et al., 2016). Um indicador de nível de serviço (*Service Level Indicator*, SLI) é uma medida quantitativa de algum aspecto do serviço, como latência, disponibilidade ou fração de requisições atendidas com sucesso. Um objetivo de nível de serviço (*Service Level Objective*, SLO) é um valor-alvo ou uma faixa aceitável para um SLI, por exemplo, atender 99% das requisições dentro de um limite de tempo. Por fim, um acordo de nível de serviço (*Service Level Agreement*, SLA) é o contrato entre provedor e cliente que formaliza esses objetivos e define as consequências, geralmente financeiras, de não cumpri-los.

Na literatura de alocação de recursos, a violação de SLA é frequentemente aproximada por medidas derivadas da utilização, já que os rastros públicos não registram a latência percebida pelos usuários. Beloglazov e Buyya (2012), por exemplo, medem a fração do tempo em que os servidores ativos operam com utilização máxima, situação em que o desempenho das aplicações é presumivelmente degradado. Rossi et al. (2023) consideram atendida a demanda que fica abaixo da capacidade prevista.

Este trabalho adota uma aproximação do mesmo tipo. O SLI é a fração da demanda atendida pela capacidade provisionada, e ocorre uma violação no intervalo t quando a demanda real supera a capacidade disponível, isto é, quando $d_t > k \cdot n_t$, na notação da Equação 1-1. O parâmetro ε daquela equação faz o papel do SLO. Essa simplificação não captura efeitos como filas ou degradação gradual de desempenho perto da saturação, mas permite comparar estratégias de alocação de forma consistente e reproduzível.

2.1.3 Provisionamento reativo e preditivo

O escalonamento automático (*autoscaling*) é o mecanismo que ajusta os recursos alocados a uma aplicação sem intervenção humana. Ele pode ser horizontal, quando altera o número de instâncias (máquinas virtuais ou contêineres), ou vertical, quando altera a capacidade de cada instância. Este trabalho trata apenas do escalonamento horizontal.

Lorido-Botran, Miguel-Alonso e Lozano (2014) classificam as técnicas de escalonamento automático em cinco famílias: regras baseadas em limiares, aprendizado por reforço, teoria de filas, teoria de controle e análise de séries temporais. Do ponto de vista deste trabalho, a distinção mais importante é entre abordagens reativas e preditivas. No provisionamento reativo, a decisão no instante t é tomada a partir do que já foi observado, normalmente a utilização corrente. No provisionamento preditivo, a decisão é tomada a partir de uma estimativa $\hat{d}_{t+h}$ da demanda futura.

A desvantagem da abordagem reativa é o atraso. Entre a detecção de um aumento de carga e a disponibilidade de uma nova instância há o tempo de inicialização, que não é desprezível e varia entre provedores e configurações (MAO; HUMPHREY, 2012). Durante esse intervalo, a aplicação opera com capacidade insuficiente. A abordagem preditiva tenta antecipar a demanda com um horizonte h pelo menos igual a esse tempo de inicialização, de modo que a instância já esteja disponível quando a carga chegar. Provedores comerciais já oferecem modos de escalonamento preditivo, como o do Amazon EC2 Auto Scaling (Amazon Web Services, 2024), o que reforça a relevância prática do problema.

Qualquer que seja a abordagem, cada decisão pode errar em duas direções. Há superprovisionamento quando a capacidade alocada excede a demanda ($k \cdot n_t > d_t$): o serviço é atendido, mas a capacidade ociosa $k \cdot n_t - d_t$ representa custo sem retorno. Há subprovisionamento quando a capacidade é menor que a demanda ($k \cdot n_t < d_t$): o custo é menor, mas parte da demanda não é atendida e o SLA pode ser violado. O objetivo da alocação é equilibrar essas duas falhas, e a qualidade da previsão determina o quanto esse equilíbrio pode ser alcançado.

2.1.4 Métricas de avaliação

As métricas usadas na literatura se dividem em dois grupos, e a distinção entre eles é central para este trabalho.

Métricas de previsão. Medem o quanto a previsão se aproxima do valor real, independentemente de como ela será usada. Para N previsões $\hat{y}_i$ e valores reais y_i, as mais comuns são:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2, \qquad \text{RMSE} = \sqrt{\text{MSE}}, \tag{2-1}$$

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|, \qquad \text{MAPE} = \frac{100\%}{N}\sum_{i=1}^{N}\left|\frac{y_i - \hat{y}_i}{y_i}\right|. \tag{2-2}$$

O MAPE é indefinido quando algum y_i é zero e instável quando y_i é próximo de zero, situação comum em períodos de baixa utilização. Além disso, as quatro métricas são simétricas: superestimar e subestimar a demanda pela mesma quantidade produz o mesmo erro. Para a alocação, porém, esses erros têm consequências diferentes, já que a superestimação gera custo e a subestimação gera violações. Por isso, um modelo com menor RMSE não é necessariamente o que leva às melhores decisões de alocação.

Métricas de alocação. Medem o resultado das decisões tomadas a partir da previsão. Para uma simulação com T intervalos, as métricas adotadas neste trabalho são:

$$C = \sum_{t=1}^{T} c \cdot n_t \qquad \text{(custo operacional total)} \tag{2-3}$$

$$V = \frac{1}{T}\sum_{t=1}^{T} \mathbf{1}\left[\, d_t > k \cdot n_t \,\right] \qquad \text{(fração de intervalos com violação)} \tag{2-4}$$

$$U = \frac{\sum_t \max(0,\, d_t - k \cdot n_t)}{\sum_t d_t} \qquad \text{(fração da demanda não atendida)} \tag{2-5}$$

$$O = \frac{\sum_t \max(0,\, k \cdot n_t - d_t)}{\sum_t k \cdot n_t} \qquad \text{(fração da capacidade ociosa)} \tag{2-6}$$

Também é útil contar o número de alterações em n_t ao longo da simulação, pois políticas que oscilam com frequência são indesejáveis na prática. Essas métricas correspondem diretamente aos termos da formulação da Equação 1-1: C é a função objetivo e V é o lado esquerdo da restrição.

As métricas de nível de serviço propostas por Rossi et al. (2023) ficam entre os dois grupos. A taxa de sucesso mede a fração de intervalos em que a demanda real fica abaixo do limite superior da previsão, e o total de recursos previstos soma esses limites superiores em relação à demanda real. Ambas são calculadas sobre a própria previsão, mas se aproximam, respectivamente, do complemento de V e do custo C no caso em que a capacidade pode ser ajustada

de forma contínua. Este trabalho dá um passo adiante ao aplicar a previsão a uma política de alocação com instâncias discretas e medir as métricas de alocação diretamente.

2.2 Abordagens para previsão de carga

2.2.1 Modelos estatísticos

Os métodos mais simples de previsão servem como referência para todos os demais (HYNDMAN; ATHANASOPOULOS, 2018). O método ingênuo, ou de persistência, assume que o próximo valor será igual ao último observado, $\hat{y}_{t+1} = y_t$. A média móvel usa a média das w últimas observações. A suavização exponencial simples atribui pesos que decaem geometricamente com a idade da observação:

$$\hat{y}_{t+1} = \alpha\, y_t + (1-\alpha)\, \hat{y}_t, \qquad 0 < \alpha \leq 1, \tag{2-7}$$

em que valores maiores de α fazem o modelo reagir mais rapidamente a mudanças. O método de Holt-Winters estende essa ideia com componentes de tendência e de sazonalidade, o que o torna adequado a séries com padrões periódicos, como os ciclos diários comuns em cargas de nuvem (HYNDMAN; ATHANASOPOULOS, 2018).

A família ARIMA (*AutoRegressive Integrated Moving Average*), sistematizada pela metodologia de Box e Jenkins (BOX et al., 2015), é o modelo estatístico mais usado em previsão de séries temporais. Um modelo ARIMA(p, d, q) aplica d diferenciações para tornar a série estacionária e descreve o resultado como combinação linear de p valores passados (componente autorregressiva) e de q erros de previsão passados (componente de média móvel). Usando o operador de defasagem B, definido por $B\, y_t = y_{t-1}$, o modelo é escrito como:

$$\left(1 - \sum_{i=1}^{p} \phi_i B^i\right) (1-B)^d\, y_t = \beta_0 + \left(1 + \sum_{j=1}^{q} \theta_j B^j\right) a_t, \tag{2-8}$$

em que ϕ_i e θ_j são os coeficientes estimados, β_0 é uma constante e a_t é um ruído branco. A variante sazonal, SARIMA, acrescenta termos equivalentes defasados pelo período da sazonalidade. A escolha das ordens p, d e q pode ser feita pela análise das funções de autocorrelação ou por critérios de informação, como o AIC (HYNDMAN; ATHANASOPOULOS, 2018).

No contexto de nuvem, Calheiros et al. (2015) aplicaram um modelo ARIMA a rastros reais de requisições a servidores *web* para apoiar o provisionamento proativo de recursos de provedores de *software* como serviço. Além

da precisão da previsão, que chegou a 91% em média, os autores avaliaram o efeito das previsões sobre a utilização de recursos e a qualidade de serviço das aplicações, ligando explicitamente a precisão da previsão às consequências para o serviço.

Limitações. Os modelos estatísticos clássicos assumem relações lineares e, no caso do ARIMA, estacionariedade após a diferenciação. Cargas de nuvem frequentemente violam essas hipóteses, com mudanças abruptas de regime e picos que não seguem um padrão linear. Esses modelos também são essencialmente univariados em sua forma básica e precisam ser reajustados quando o comportamento da série muda. Por outro lado, eles exigem poucos dados, são baratos de ajustar e têm parâmetros interpretáveis. Por isso, devem estar presentes em qualquer comparação: uma rede neural só se justifica se superar essas alternativas mais simples sob as mesmas condições.

2.2.2 Redes neurais recorrentes

O esDNN (XU et al., 2022) é uma abordagem multivariada que combina uma GRU modificada com camadas convolucionais. Os autores convertem as séries de uso de processador e de memória em um problema de aprendizado supervisionado por meio de uma janela deslizante e avaliam o modelo nos rastros da Alibaba e do Google, comparando-o com RNN, GRU e Bi-LSTM em horizontes que vão de segundos a dias. Os resultados indicam que a GRU pura é competitiva em horizontes curtos, enquanto o esDNN mantém erro menor e mais estável em horizontes longos. Os autores também integram as previsões a uma simulação de escalonamento automático de máquinas físicas, com período de decisão de cinco minutos e limiar de utilização de 80%.

2.2.3 Modelos com quantificação de incerteza

Rossi et al. (2023) propõem modelos bayesianos profundos (HBNN e LSTMD), em versões univariada e bivariada, que preveem não apenas um valor, mas uma distribuição para a demanda futura de processador e de memória. Os experimentos usam rastros do Google e da Alibaba e comparam os modelos com uma LSTM determinística. Os autores mostram que as métricas de erro pontual não distinguem bem os modelos, enquanto métricas de nível de serviço, calculadas sobre o limite superior do intervalo de confiança, evidenciam a vantagem de modelar a incerteza. O trabalho também investiga transferência de aprendizado e conclui que ela funciona razoavelmente dentro de um mesmo provedor, mas se degrada entre provedores diferentes.

2.3 Abordagens para alocação de recursos

2.3.1 Heurísticas e políticas reativas

As políticas baseadas em limiares são a forma mais difundida de escalonamento automático, sobretudo pela simplicidade (LORIDO-BOTRAN; MIGUEL-ALONSO; LOZANO, 2014). Na forma mais comum, define-se um limiar superior θ_{sup} e um inferior θ_{inf} para a utilização média das instâncias: se a utilização permanece acima de θ_{sup} por um certo número de intervalos, uma instância é adicionada; se permanece abaixo de θ_{inf}, uma instância é removida. Um período de espera (*cooldown*) após cada ação impede que o sistema oscile.

Uma variação proporcional é usada pelo *Horizontal Pod Autoscaler* do Kubernetes, que calcula o número desejado de réplicas a partir da razão entre a métrica observada u_t e um valor-alvo u^* (The Kubernetes Authors, 2024):

$$n_{t+1} = \left\lceil n_t \cdot \frac{u_t}{u^*} \right\rceil . \tag{2-9}$$

O mecanismo inclui ainda uma faixa de tolerância, dentro da qual nenhuma ação é tomada, e uma janela de estabilização para reduções, ambas destinadas a evitar oscilações. Outro exemplo de política reativa é a usada como referência pelo esDNN, que define o número de máquinas ativas como a média do número de máquinas dos cinco intervalos anteriores (XU et al., 2022).

Um problema relacionado, mas distinto, é o posicionamento das instâncias nos servidores físicos. Ele é tratado como uma variante do problema de empacotamento (*bin packing*), que é NP-difícil, e por isso costuma ser resolvido com heurísticas como *First-Fit* e *Best-Fit Decreasing*. Beloglazov e Buyya (2012), por exemplo, adaptam esta última para consolidar máquinas virtuais e reduzir o consumo de energia. Meta-heurísticas como algoritmos genéticos, otimização por enxame de partículas e colônia de formigas exploram o espaço de soluções de forma mais ampla, ao custo de maior tempo de computação (BODRA; KHAIRNAR, 2025). Como definido na Seção 1.4, o posicionamento está fora do escopo deste trabalho, que decide apenas quantas instâncias manter ativas.

Limitações. Por construção, as políticas reativas só respondem depois que a mudança de carga já ocorreu, e o tempo de inicialização das instâncias amplia esse atraso. Os limiares precisam ser ajustados manualmente para cada aplicação: limiares agressivos causam oscilações, e limiares conservadores mantêm capacidade ociosa. Ainda assim, essas políticas são simples, previsíveis e amplamente usadas, o que faz da política reativa por limiar uma referência

obrigatória na avaliação deste trabalho.

2.3.2 Alocação guiada por previsão

Na alocação guiada por previsão, a decisão é tomada em duas etapas desacopladas. Primeiro, um modelo estima a demanda $\hat{d}_{t+h}$ para o intervalo em que a decisão terá efeito. Em seguida, uma regra converte essa estimativa em número de instâncias. A regra mais direta divide a demanda prevista pela capacidade de uma instância, acrescentando uma margem de segurança $m \geq 0$:

$$n_{t+h} = \left\lceil \frac{(1+m)\,\hat{d}_{t+h}}{k} \right\rceil . \tag{2-10}$$

Quando o modelo fornece uma distribuição de probabilidade em vez de um valor pontual, a margem pode ser substituída por um quantil: em vez de $(1+m)\,\hat{d}_{t+h}$, usa-se o valor q_τ abaixo do qual a demanda fica com probabilidade τ. É o que fazem Rossi et al. (2023) ao usar o limite superior do intervalo de confiança com níveis de serviço-alvo entre 90% e 99,5%.

A margem m, ou o quantil τ, é o parâmetro que controla o equilíbrio entre custo e violações. Margens maiores reduzem V e aumentam C, e o contrário vale para margens menores. Na formulação da Equação 1-1, escolher esse parâmetro é a forma prática de buscar o menor custo que ainda respeita a fração de violações ε, e essa escolha pode ser calibrada em um conjunto de validação. Além disso, o horizonte h deve ser compatível com o tempo de inicialização das instâncias, pois uma previsão para o intervalo seguinte é inútil se a instância leva mais do que um intervalo para ficar disponível (MAO; HUMPHREY, 2012).

O esDNN (XU et al., 2022) é um exemplo dessa abordagem: as previsões de uso de processador alimentam uma simulação de escalonamento automático de máquinas físicas com período de decisão de cinco minutos e limiar de utilização de 80%. A avaliação, no entanto, concentra-se na redução do número de máquinas ativas em relação à política reativa por média, sem medir violações de SLA nem comparar diferentes preditores sob a mesma regra de decisão.

A principal vantagem do desacoplamento é a modularidade. Como a regra de decisão é fixa, é possível trocar o modelo de previsão mantendo todo o resto constante, o que permite atribuir as diferenças de custo e de violações exclusivamente à qualidade da previsão. É essa propriedade que este trabalho explora para comparar redes neurais e métodos tradicionais.

2.3.3 Aprendizado por reforço

A revisão de Bodra e Khairnar (2025) compara abordagens recentes de aprendizado de máquina para alocação de recursos em nuvem, incluindo métodos de aprendizado por reforço profundo, meta-heurísticas híbridas e modelos de previsão. Os autores observam que os trabalhos analisados são avaliados em ambientes, conjuntos de dados e métricas muito diferentes entre si, muitas vezes com cargas sintéticas, o que dificulta a comparação direta. Apontam ainda que os métodos de aprendizado por reforço profundo são sensíveis ao projeto da função de recompensa e aos hiperparâmetros e enfrentam limitações de escalabilidade conforme o espaço de estados cresce.

No aprendizado por reforço, a alocação é formulada como um problema de decisão sequencial. Um agente observa o estado do sistema, como a utilização recente e o número de instâncias ativas, escolhe uma ação, como adicionar, remover ou manter instâncias, e recebe uma recompensa que penaliza o custo e as violações de SLA. Ao longo de muitas interações, o agente aprende uma política que maximiza a recompensa acumulada. Diferentemente da alocação guiada por previsão, previsão e decisão não são separadas: a política mapeia diretamente estados em ações, e a antecipação da demanda fica implícita no que o agente aprende. O aprendizado por reforço aparece tanto na classificação de Lorido-Botran, Miguel-Alonso e Lozano (2014) quanto na revisão de Bodra e Khairnar (2025).

Limitações e posicionamento. Além da sensibilidade à função de recompensa e aos hiperparâmetros apontada por Bodra e Khairnar (2025), uma política aprendida é difícil de interpretar: quando ela toma uma decisão ruim, não é simples identificar se o problema está na antecipação da demanda ou na regra de decisão. O treinamento também exige um ambiente de simulação e um grande número de episódios. Por esses motivos, este trabalho adota a abordagem desacoplada da Seção 2.3.2, que permite isolar e medir o efeito da previsão sobre a alocação, e deixa o aprendizado por reforço fora do escopo (Seção 1.4). Comparar a abordagem desacoplada com uma política aprendida é uma extensão natural para trabalhos futuros.

2.4 Comparativo dos trabalhos relacionados

A Tabela 2.1 resume os trabalhos mais próximos desta proposta segundo os critérios relevantes para o problema definido na Seção 1.2.

Tabela 2.1: Comparativo entre os trabalhos relacionados e a proposta deste trabalho.

Critério	**Xu et al. (2022)**	**Rossi et al. (2023)**	**Bodra e Khairnar (2025)**	**Este trabalho**
Tipo	Método de previsão	Método de previsão	Revisão comparativa	Método e avaliação
Técnica	GRU modificada com CNN, multivariada	Redes bayesianas (HBNN, LSTMD)	DRL, meta-heurísticas e previsores	Rede neural recorrente
Dados	Alibaba e Google	Alibaba e Google	Variados, muitos sintéticos	Alibaba e Google
Métricas	MSE, RMSE, MAPE	Erro pontual e nível de serviço	Heterogêneas entre trabalhos	Erro de previsão, custo e violações de SLA
Acopla previsão à alocação	Parcial (simulação de autoscaling)	Não	Não se aplica	Sim
Compara preditores sob a mesma política	Não	Não	Não	Sim

2.4.1 Síntese e lacuna

2.5 Tecnologias Utilizadas

Esta seção descreve as linguagens, bibliotecas e ferramentas escolhidas para o desenvolvimento do projeto, organizadas conforme as etapas do pipeline: tratamento dos dados, previsão de carga, simulação da alocação de recursos e ambiente de execução.

2.5.1 Linguagens de Programação

O projeto utiliza duas linguagens. Python é empregada nas etapas de tratamento de dados e previsão de carga, em razão da ampla disponibilidade de bibliotecas maduras para séries temporais e aprendizado profundo. Java é

empregada na etapa de simulação, por ser a linguagem do simulador de nuvem adotado (CloudSim Plus).

2.5.2 Tratamento dos Dados

Os rastros de utilização de recursos (*cluster traces*) serão processados com a biblioteca *pandas* (MCKINNEY, 2010), que permite agregar os registros brutos em séries temporais de carga, com granularidade temporal definida de acordo com as necessidades da previsão. As mesmas séries alimentam tanto os modelos estatísticos quanto as redes neurais, o que garante a comparabilidade entre as abordagens.

2.5.3 Modelos de Previsão

Os métodos tradicionais de previsão, usados como *baselines*, serão implementados com a biblioteca *statsmodels* (SEABOLD; PERKTOLD, 2010), que oferece os modelos ARIMA e de suavização exponencial. A rede neural proposta será desenvolvida e treinada com *PyTorch* (PASZKE et al., 2019), que oferece flexibilidade para definir arquiteturas recorrentes e personalizar o ciclo de treinamento.

2.5.4 Simulação

A avaliação da alocação proativa de recursos será realizada no simulador CloudSim Plus (FILHO et al., 2017), um *framework* em Java para simulação de ambientes de computação em nuvem. Ele permite modelar *data centers*, máquinas físicas e virtuais, e políticas de alocação, além de possuir suporte nativo à reprodução de rastros do Google Cluster Data. Isso viabiliza a comparação das políticas de alocação, com e sem previsão, sobre uma mesma carga de trabalho.

2.5.5 Ambiente de Desenvolvimento

O treinamento dos modelos é executado em máquina local. O versionamento do código é feito com Git, e o ambiente de execução é encapsulado em contêineres Docker, para facilitar a reprodutibilidade dos experimentos.

3
Referências bibliográficas

Alibaba Group. **Alibaba Cluster Trace Program**. 2018. Disponível em: https://github.com/alibaba/clusterdata.

Amazon Web Services. **Predictive scaling for Amazon EC2 Auto Scaling**. 2024. Documentação do Amazon EC2 Auto Scaling. Disponível em: https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-predictive-scaling.html. Acesso em: 14 out. 2026.

BELOGLAZOV, A.; BUYYA, R. Optimal online deterministic algorithms and adaptive heuristics for energy and performance efficient dynamic consolidation of virtual machines in cloud data centers. **Concurrency and Computation: Practice and Experience**, v. 24, n. 13, p. 1397–1420, 2012.

BEYER, B. et al. **Site Reliability Engineering: How Google Runs Production Systems**. Sebastopol, CA: O'Reilly Media, 2016.

BODRA, D.; KHAIRNAR, S. Machine learning-based cloud resource allocation algorithms: a comprehensive comparative review. **Frontiers in Computer Science**, v. 7, p. 1678976, 2025.

BOX, G. E. P. et al. **Time Series Analysis: Forecasting and Control**. 5. ed. Hoboken, NJ: John Wiley & Sons, 2015.

CALHEIROS, R. N. et al. Workload prediction using ARIMA model and its impact on cloud applications' QoS. **IEEE Transactions on Cloud Computing**, v. 3, n. 4, p. 449–458, 2015.

FILHO, M. C. S. et al. CloudSim Plus: a cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness. In: **2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)**. [S.l.: s.n.], 2017. p. 400–406.

HERBST, N. R.; KOUNEV, S.; REUSSNER, R. Elasticity in cloud computing: What it is, and what it is not. In: **Proceedings of the 10th International Conference on Autonomic Computing (ICAC 13)**. San Jose, CA: USENIX, 2013. p. 23–27.

HYNDMAN, R. J.; ATHANASOPOULOS, G. **Forecasting: principles and practice**. 2. ed. Melbourne: OTexts, 2018. Disponível em: https://otexts.com/fpp2/.

LORIDO-BOTRAN, T.; MIGUEL-ALONSO, J.; LOZANO, J. A. A review of auto-scaling techniques for elastic applications in cloud environments. **Journal of Grid Computing**, v. 12, n. 4, p. 559–592, 2014.

MAO, M.; HUMPHREY, M. A performance study on the VM startup time in the cloud. In: **2012 IEEE Fifth International Conference on Cloud Computing (CLOUD)**. [S.l.: s.n.], 2012. p. 423–430.

MCKINNEY, W. Data structures for statistical computing in Python. In: WALT, S. van der; MILLMAN, J. (Ed.). **Proceedings of the 9th Python in Science Conference**. [S.l.: s.n.], 2010. p. 56–61.

MELL, P.; GRANCE, T. **The NIST Definition of Cloud Computing**. Gaithersburg, MD, 2011.

PASZKE, A. et al. PyTorch: An imperative style, high-performance deep learning library. In: **Advances in Neural Information Processing Systems 32 (NeurIPS 2019)**. [S.l.: s.n.], 2019.

ROSSI, A. et al. **Forecasting Workload in Cloud Computing: Towards Uncertainty-Aware Predictions and Transfer Learning**. 2023. Código e dados disponíveis em: https://github.com/andreareds/TowardsUncertaintyAwareWorkloadPrediction.

SEABOLD, S.; PERKTOLD, J. statsmodels: Econometric and statistical modeling with Python. In: **Proceedings of the 9th Python in Science Conference**. [S.l.: s.n.], 2010.

The Kubernetes Authors. **Horizontal Pod Autoscaling**. 2024. Documentação do Kubernetes. Disponível em: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/. Acesso em: 14 out. 2026.

WILKES, J. **Google cluster-usage traces v3**. Mountain View, CA, USA, 2020. Disponível em: https://github.com/google/cluster-data.

XU, M. et al. esDNN: Deep neural network based multivariate workload prediction approach in cloud environments. **ACM Transactions on Internet Technology**, 2022. Disponível em: https://arxiv.org/abs/2203.02684.